

Projet de Software Engineering 2025 (Java)

Data compressing for speed up transmission

Auteur :
Bensallah Younes

Université Côte d'Azur

Master 1 Informatique

Année universitaire 2025–2026

Date rendu : 2 Novembre 2025

Dépôt GitHub : `Software Engineering Project 2025`

Contents

1 Introduction **2**

2 Modes de compression implémentés **3**

2.1 Mode OVERLAP (avec chevauchement) 3

2.2 Mode NO_OVERLAP (sans chevauchement) 4

2.3 Mode OVERFLOW (avec zone de débordement) 4

3 Architecture du projet **5**

3.1 Design général et Pattern utilisé 5

3.2 Structure interne de la classe `BitPacking` 6

3.3 Séparation des responsabilités 6

3.4 Classe `Main` : interface d'exécution 7

4 Méthodologie des benchmarks **7**

4.1 Objectifs des mesures 7

4.2 Protocole de mesure 8

4.3 Jeux de tests utilisés 8

4.4 Seuil de rentabilité de la compression 9

4.5 Plateforme d'exécution 9

5 Analyse des performances **9**

5.1 Tableau récapitulatif 10

5.2 Lecture des résultats 10

6 Limites et perspectives **11**

6.1 Extension possible : gestion des valeurs négatives 12

7 Conclusion **12**

1 Introduction

La transmission rapide et efficace des données est une question primordial en programmation. Réduire la taille de ces données avant leur envoi permet alors de diminuer le temps de transmission et d'économiser la bande passante, au prix d'un surcoût de calcul lié à la compression et à la décompression.

Le projet s'inscrit alors dans cette question, il nous a été demandé d'implémenter trois modes différents de *Bit Packing*, qui est une méthode de compression sans perte regroupant les entiers selon le nombre minimal de bits nécessaires. Si chaque entier peut être codé sur k bits, on peut ainsi représenter n valeurs sur $n \times k$ bits au lieu de $32n$ bits utilisés par les int en Java.

L'objectif est par conséquent double :

- réduire la taille totale à transmettre tout en conservant un accès direct à chaque élément (fonction `get(i)`),
- comparer différentes stratégies de compression en termes de rapidité et d'efficacité.

Trois variantes de la méthode ont été développées (comme demandé) :

1. **OVERLAP (chevauchement)** : les valeurs compressées peuvent s'étendre sur deux mots consécutifs de 32 bits
2. **NO_OVERLAP (sans chevauchement)** : chaque valeur reste confinée à un seul mot
3. **OVERFLOW (débordement)** : une extension optimisée qui déplace les valeurs trop grandes dans une zone de débordement afin d'éviter de gaspiller des bits

Chaque mode a été implémenté dans une classe unique `BitPacking`, conçue selon le design pattern de *factory* permettant d'instancier la version désirée à partir d'un seul paramètre. Afin de comparer tout cela, un ensemble d'expériences et de mesures de performances a alors été réalisé pour évaluer :

- le temps moyen de compression et de décompression.
- le temps d'accès à un élément compressé (`get(i)`).
- le seuil de bande passante B^* à partir duquel la compression devient réellement rentable.

Ce rapport présente dans un premier temps les principes du *Bit Packing* et la conception de l'architecture logicielle retenue. Les trois méthodes de compression sont ensuite détaillées, avant d'exposer la méthodologie de mesure, les résultats expérimentaux et une discussion sur les performances et la rentabilité de chaque approche.

2 Modes de compression implémentés

Dans cette partie, nous verrons les trois modes de compression implémentés : **OVERLAP**, **NO_OVERLAP** et **OVERFLOW**. Chaque mode répond à une logique différente et possède ses propres avantages, selon la structure des données à compresser.

Ces trois approches partagent la même idée de base : représenter un tableau d'entiers sur le nombre minimal de bits nécessaire à chaque valeur. Le choix du mode influe directement sur la manière dont ces bits sont répartis dans les mots de 32 bits, ainsi que sur la vitesse des opérations de lecture et d'écriture.

Avant de rentrer dans le détail de chaque technique, notons que :

- les valeurs doivent être non négatives (sinon une exception est levée),
- la taille nécessaire en bits est calculée via `computeK()`,
- la décompression ne dépend jamais du tableau original mais uniquement du buffer compressé,
- l'accès aléatoire `get(i)` reste en temps constant dans les trois modes.

Ces points garantissent que cela reste efficace et fidèle à la consigne du projet.

2.1 Mode OVERLAP (avec chevauchement)

Le mode **OVERLAP** correspond à la version la plus fidèle à l'idée brute du bit-packing. On écrit ici les valeurs dans un flux de bits continu, même si cela implique qu'une valeur puisse être répartie entre deux mots de 32 bits.

La logique est la suivante, on maintient un pointeur de bit global (`bitPtr`) et on écrit séquentiellement les valeurs. Si une valeur déborde du mot courant, la partie restante est écrite dans le suivant. Ce comportement est géré dans `packOverlap()`, à l'aide de décalages binaires et de masques (`mask()`).

Ce mode est donc le plus compact possible, car aucun bit n'est gaspillé. Cependant, du fait que l'on doit éventuellement assembler des fragments situés sur deux mots, cela peut etre assez couteux.

2.2 Mode NO_OVERLAP (sans chevauchement)

Dans ce mode, chaque valeur doit tenir dans un seul mot de 32 bits. On adopte alors une logique par blocs, on calcule d'abord combien de valeurs tiennent dans un mot (`vpw (values per word) = 32 / k`) puis on range les valeurs proprement, sans jamais déborder.

La méthode `packNoOverlap()` applique cette idée. Comme aucune valeur ne traverse de frontière entre deux mots, l'extraction `get(i)` est extrêmement simple et rapide, un décalage et un masque suffisent alors.

Ce mode est donc logiquement plus rapide, mais potentiellement légèrement moins compact car il peut rester des bits inutilisés dans chaque mot.

2.3 Mode OVERFLOW (avec zone de débordement)

Le troisième mode est, à mon sens, le plus intéressant conceptuellement. Il vise les tableaux contenant essentiellement de petites valeurs, mais aussi quelques valeurs très grandes qui viendraient sinon imposer un `k` trop élevé.

L'idée est donc la suivante :

- on cherche un *seuil optimal* k' pour coder la majorité des valeurs
- les valeurs nécessitant plus de bits que k' sont envoyées dans une zone de *débordement*
- chaque valeur est précédée d'un **flag** :
  - 0 : la valeur suit directement (sur k' bits)
  - 1 : la valeur est dans la zone overflow

La méthode `packOverflow()` teste plusieurs valeurs possibles pour k' et retient celle offrant la meilleure possibilité en termes de bits utilisés. Pour localiser rapidement les valeurs débordées, un tableau `prefixOv[]` est maintenu afin de savoir combien d'éléments overflow précèdent un index donné.

Ce mode permet donc d'obtenir un taux de compression optimal sur des distributions hétérogènes, tout en conservant un accès direct en O(1) à chaque valeur (via l'index overflow).

Nous verrons par la suite, lors de l'explication des mesures, des comparaisons entre chaque mode permettant alors de determiner le quel est le plus efficace dans quel cas. Penchons nous à présent plus en détail sur notre implémentation.

3 Architecture du projet

Dans cette partie, nous allons présenter l'architecture générale du projet ainsi que les choix techniques réalisés pour structurer le code. L'objectif était d'obtenir une solution claire, extensible et respectant les contraintes du sujet.

Le fichier principal du projet est la classe `BitPacking`, qui contient l'intégralité de la logique de compression, de stockage et de décompression des données. Afin de simplifier l'exécution et les tests, une classe `Main` vient en support pour charger un fichier d'entrée, exécuter la compression et afficher les résultats.

3.1 Design général et Pattern utilisé

Le projet suit une approche classique en orientée objet, avec quelques points importants :

- une **seule classe** `BitPacking` qui implémente les trois modes
- une **factory method** permettant de créer une instance selon le mode souhaité
- un accès unifié via les méthodes `compress()`, `get()` `decompress()`

Le pattern de **factory** est utilisé via :

```
public static BitPacking of(Mode mode) {
    return new BitPacking(mode);
}
```

Ce choix permet d'instancier facilement un mode de compression uniquement à partir d'un paramètre, tout en gardant une seule structure interne et donc un code plus compact et réutilisable.

3.2 Structure interne de la classe `BitPacking`

La classe possède plusieurs champs permettant de conserver l'état de la compression :

- `mode` : le mode choisi (OVERLAP, NO_OVERLAP ou OVERFLOW)
- `n` : taille du tableau original
- `k` : nombre de bits nécessaires par valeur
- `words[]` : tableau compressé de 32 bits

Dans le cas du mode OVERFLOW, des variables supplémentaires sont utilisées pour gérer la zone de débordement :

- `kPrime` : taille en bits des valeurs "normales"
- `kOverflow` : taille en bits des valeurs débordées
- `prefixOv[]` : tableau pour compter les débordements avant une position donnée
- `bitsIdx` : nombre de bits nécessaires pour indexer la zone overflow
- `overflowStartBit` : position du début de cette zone dans le flux binaire

Ce découpage permet de séparer clairement les responsabilités internes et de garantir l'accès constant à chaque élément, comme demandé dans la consigne.

3.3 Séparation des responsabilités

Les principales étapes de traitement sont séparées en méthodes dédiées :

- `compress()` : point d'entrée, choisit l'algorithme selon le mode
- `packOverlap()` : écriture avec chevauchement
- `packNoOverlap()` : écriture sans chevauchement
- `packOverflow()` : écriture avec débordement optimisé
- `get(i)` : récupération directe de la valeur à l'indice i
- `decompress()` : reconstitue le tableau original

L'ensemble des opérations binaires (décalages, masques, calculs de bits nécessaires) restent encapsulées dans la classe, ce qui évite toute fuite de logique interne.

3.4 Classe `Main` : interface d'exécution

La classe `Main` sert uniquement à exécuter les tests et afficher les résultats. Elle permet de :

- charger un fichier contenant des entiers
- sélectionner un mode via la ligne de commande
- exécuter la compression et la décompression
- afficher les temps et les tailles obtenus

Ce choix permet de séparer le BitPacking de la logique de test/exécution.

En conclusion, cette architecture rend le projet clair, modulaire et facilement extensible. Il serait par exemple très simple d'ajouter un quatrième mode de compression sans modifier l'interface publique du programme, uniquement en ajoutant une méthode de traitement interne. Intéressons nous maintenant à nos benchmarks.

4 Méthodologie des benchmarks

Dans cette partie, nous verrons la manière dont les performances ont été mesurées afin de comparer efficacement les trois modes de compression. L'objectif est d'obtenir des résultats fiables et représentatifs de l'utilisation réelle du programme.

4.1 Objectifs des mesures

Nous avons mesuré plusieurs aspects importants :

- le temps moyen pour compresser un tableau,
- le temps moyen pour le décompresser,
- le temps moyen d'accès aléatoire à une valeur via `get(i)`,
- la taille finale obtenue en bits,
- le seuil de bande passante à partir duquel la compression devient rentable.

Ces aspects permettent de déterminer non seulement l'efficacité en termes de taille, mais aussi le coût en calcul et en accès mémoire.

4.2 Protocole de mesure

Pour garantir des résultats fiables, plusieurs précautions ont été prises :

- chaque opération est exécutée plusieurs fois, puis on récupère la moyenne
- un **warm-up** est effectué avant la mesure réelle afin d'éviter les effets de cache
- les temps sont mesurés avec `System.nanoTime()` pour une meilleure précision.

Concrètement, la méthode utilisée pour mesurer par exemple la compression est la suivante :

```
timeCompressAvgNs(mode, src, warmup = 5, runs = 20)
```

Ce qui signifie :

- 5 compressions effectuées en "échauffement" afin d'éviter les effets de cache
- puis 20 compressions réellement comptées
- la moyenne est ensuite retournée

Le même principe est appliqué pour la décompression et pour la mesure moyenne de `get(i)`.

4.3 Jeux de tests utilisés

Les tests ont été effectués sur des fichiers contenant des valeurs entières. Plusieurs distributions ont été utilisées afin d'observer le comportement selon la nature des données :

- valeurs petites et homogenes (ex : beaucoup de petits entiers)
- valeurs hétérogenes avec pics occasionnels (cas idéal pour `OVERFLOW`)
- valeurs aléatoires

Cela permet d'obtenir des conclusions plus générales et d'éviter un biais lié à un seul type de données.

4.4 Seuil de rentabilité de la compression

La consigne du projet demande de calculer le seuil de bande passante B^* à partir duquel la compression devient intéressante.

Ce seuil est calculé avec la formule suivante déjà fournie dans le code :

$$B^* = \frac{bits économisés}{tempsCPU(compression + décompression)}$$

Plus ce seuil est bas, plus la méthode est efficace sur des réseaux lents. Si au contraire il est très élevé, alors compresser n'a d'intérêt que sur des réseaux très rapides.

4.5 Plateforme d'exécution

Les mesures ont été réalisées sur la machine suivante :

- Ubuntu 22.04.1 LTS (64-bit) (Sur Windows 11)
- Ryzen 7 3700X
- 16 Go RAM
- OpenJDK 11.0.28 (build 11.0.28+6-post-Ubuntu-1ubuntu122.04.1)

Ce point est important car les performances peuvent varier selon le matériel.

Cette méthodologie permet donc d'obtenir des mesures cohérentes, fiables et comparables entre les différents modes de compression. Les résultats obtenus seront présentés dans la section suivante.

5 Analyse des performances

Afin d'évaluer concrètement l'efficacité des trois modes de compression implémentés, une série de benchmarks systématiques est alors mise en place. L'idée était d'observer le comportement de chaque méthode dans plusieurs situations réalistes : valeurs très petites, valeurs homogènes, distribution aléatoire uniforme, ou encore tableaux contenant quelques valeurs extrêmes parmi des petites.

Pour chaque test, nous mesurons alors :

- le temps moyen de compression,

- le temps moyen de décompression,
- le temps moyen d'accès séquentiel via `get(i)`,
- la taille finale des données compressées,
- le seuil B^* au-dessus duquel la compression devient rentable.

5.1 Tableau récapitulatif

Le tableau ci-dessous, obtenable dans `summary.txt` en executant `run_all.sh`, présente une synthèse sur des cas représentatifs. Celui-ci permet de mieux comparer les trois stratégies selon plusieurs jeux de données.

Table 1: Résumé des performances des trois modes de compression sur des jeux représentatifs

Dataset	**Mode**	**Taille comp. (bits)**	**t_comp (ms)**	**t_get (ns)**	**B* (Mbit/s)**
small_values	OVERLAP	300k	0.38	5	3833.8
	NO_OVERLAP	320k	0.88	8	1920.8
	OVERFLOW	400k	2.17	5	1071.3
random	OVERLAP	2.0M	0.52	6	1114.6
	NO_OVERLAP	3.2M	1.11	8	∞
	OVERFLOW	2.1M	3.04	6	310.5
95_5_distribution	OVERLAP	1.4M	0.49	4	2044.0
	NO_OVERLAP	1.6M	0.86	8	930.1
	OVERFLOW	**520k**	3.91	5	602.3
k_small_values	OVERLAP	150k	0.23	7	2177.3
	NO_OVERLAP	160k	0.33	10	1354.7
	OVERFLOW	180k	1.32	7	529.7

5.2 Lecture des résultats

Les résultats montrent clairement que chaque approche possède un domaine d'utilisation privilégié. Il n'existe pas de "meilleur" mode absolu, mais plutôt une stratégie adaptée selon la distribution des valeurs.

- **OVERLAP** offre dans la plupart des cas la meilleure compression, tout en conservant un accès relativement rapide aux données. Dans des cas comme `small_values`, il utilise presque chaque bit de manière optimale. On observe néanmoins qu'il reste légèrement plus coûteux que `NO_OVERLAP` sur certaines opérations, dû au traitement du chevauchement entre mots.

- **NO_OVERLAP** est la solution la plus simple et la plus rapide pour les accès `get(i)`. Dès que la majorité des valeurs utilisent une taille homogène, il devient très efficace. En revanche, en présence d'une distribution instable ou très variable (comme le test `random`), sa compression devient nettement moins intéressante et peut même ne rien apporter du tout — le seuil B^* devient alors infini, signifiant que compresser n'est jamais rentable dans ce cas.

- **OVERFLOW** est particulièrement performant lorsque les données sont majoritairement petites avec quelques valeurs très grandes, comme dans le test `95_5_distribution`. Dans ce scénario, il obtient la meilleure compression globale. Par contre, cela a un coût : le temps de compression augmente parfois significativement, car la structure doit analyser et gérer le débordement.

Ces résultats valident bien les avantages théoriques de chaque approche expliqués plus haut, et confirment que le choix du mode doit se faire en fonction du profil des données.

6 Limites et perspectives

Même si les résultats obtenus sont satisfaisants au vu des attentes du sujet, plusieurs limites peuvent être visibles.

Tout d'abord, l'implémentation repose exclusivement sur des entiers non négatifs. Le sujet évoque brièvement la possibilité d'étendre le schéma aux valeurs négatives, mais cela n'a pas été intégré ici afin de rester concentré sur le coeur du projet.

Egalement, le fait que les performances dépendent fortement de la distribution des valeurs du tableau. Certains modes ne sont extrememement efficaces que dans des cas bien précis :

- **OVERLAP** perd légèrement sur l'accès `get(i)` quand k devient grand,
- **NO_OVERLAP** gaspille des bits lorsque $32k$ laisse trop de place inutilisée,
- **OVERFLOW** peut devenir coûteux si trop de valeurs se retrouvent dans la zone de débordement.

Ainsi, aucun mode n'est universellement meilleur, le choix dépend grandement du jeu de donnée utilisé.

6.1 Extension possible : gestion des valeurs négatives

Comme dit plus haut, seules les valeurs positives ont été prises en charge, j'ai cependant réfléchi a une methode pour ajouter la prise en compte des valeurs négatives.

Le principal problème vient du fait que les entiers négatifs, en Java, sont représentés en complément à deux. Cela signifie que leur représentation binaire utilise toujours 32 bits, ce qui va complètement à l'encontre du principe du bit-packing (réduire le nombre de bits utilisés).

La stratégie vu en licence était alors la suivante :

- **Bit de signe séparé** : réserver 1 bit supplémentaire pour indiquer si la valeur est négative, puis stocker la valeur absolue sur le nombre minimal de bits.

Cette approche a l'avantage d'être simple à intégrer dans l'architecture existante, sans modifier l'accès direct `get(i)` ni la logique de sélection des bits. Elle représente donc une extension naturelle pour de futures améliorations du programme.

En résumé, ce projet offre une base solide et fonctionnelle pour la compression bit-packing, tout en laissant plusieurs axes d'amélioration possibles.

7 Conclusion

Ce projet m'a permis d'explorer en profondeur le principe du bit-packing et d'en mesurer concrètement les avantages et limites selon plusieurs stratégies. Nous avons implémenté trois modes distincts **OVERLAP**, **NO_OVERLAP** et **OVERFLOW** chacun répondant à un compromis spécifique entre compacité mémoire, simplicité et rapidité d'accès.

Les résultats expérimentaux montrent que :

- **OVERLAP** offre en général la meilleure taille de compression grâce à une utilisation optimale des bits ; c'est la méthode la plus compacte.
- **NO_OVERLAP** est la plus simple et la plus rapide pour l'accès aux données, au prix d'un léger gaspillage de bits. Elle reste donc très pertinente lorsque les performances CPU priment.
- **OVERFLOW** se révèle extrêmement efficace dès que le tableau contient quelques valeurs très grandes. Il permet dans ce cas une compression bien meilleure que les deux autres méthodes, malgré un coût CPU plus important.

Au-delà de l'aspect algorithmique, ce projet a également mis l'accent sur la rigueur expérimentale : mise en place d'un protocole de mesure, automatisation des tests, et analyse des temps moyens pour éviter les biais liés au cache.

Enfin, plusieurs pistes d'amélioration existent, comme la prise en charge des nombres négatifs, l'ajout d'autres variantes de compression ou encore une optimisation orientée performance.